Cholesky paralelo: cómo construimos la investigación

Recorrido local documentado hasta el 27 de septiembre de 2026. Una narración de las preguntas, los experimentos y las decisiones, acompañada por la evidencia que las sostiene.

El objetivo

Construir una factorización de Cholesky en C con MPI y determinar cuándo resulta útil repartir su trabajo. El objetivo no fue agregar optimizaciones sin límite: fue comprender cada cambio, comprobar la corrección y medir si reducía el tiempo que nos interesaba.

Cómo avanzamos

Tramo	Qué necesitábamos resolver
Fundamentos y etapa 1	Entender las dependencias y construir una referencia secuencial razonable.
Etapas 2-5	Repartir las columnas, mejorar el acceso local y contrastar left con right en MPI.
Etapas 6-8	Pasar del cálculo aislado a generar, distribuir y reunir el resultado.
Etapas 9-11	Contrastar alternativas, usar LAPACK como referencia y consolidar el candidato.
Etapas 12-14	Medir speedup y eficiencia con dos relojes, y repetir en otra sesión.

Cómo leer los números

Los tiempos se expresan en milisegundos. Cada figura identifica la campaña y el tramo cronometrado. Las barras Q1-Q3 muestran dispersión entre ejecuciones, no intervalos de confianza. No sumamos medianas de fases para obtener el total ni dividimos tiempos de campañas distintas para atribuir una mejora.

Todas las mediciones son locales en WSL sobre un Ryzen 5 2600. Los procesos MPI de estos experimentos no están en nodos distintos. El código numérico y la medición son C; Python se usa para análisis, gráficos y documentación. Hubo asistencia de IA; las decisiones aquí se apoyan en los archivos y resultados disponibles.

El recorrido termina con la repetición del 27/09: una nueva sesión permite comprobar qué tendencias se mantienen antes de pasar al clúster. Las decisiones locales siguen siendo revisables cuando cambien el tamaño del problema y el entorno de ejecución.

Fuentes principales: src/, scripts/ y resultados/ bajo desde_cero. El constructor genera un manifiesto con SHA-256 de los CSV consultados. No se ejecutaron nuevas pruebas de rendimiento para redactar este documento.

Fundamentos: qué podemos calcular en paralelo

Para una matriz simétrica definida positiva buscamos un factor triangular inferior L. Ambas variantes calculan la misma factorización; cambia cuándo reúnen o propagan las contribuciones.

$$A = LL^T$$

Left-looking: reunir lo que ya sabemos

Al construir la columna j usamos los resultados de las columnas anteriores. Una vez disponibles esas contribuciones y el pivote, se pueden calcular en paralelo diferentes elementos debajo de la diagonal. También se pueden repartir sumas parciales sobre columnas anteriores y luego combinarlas, que es lo que probamos en nuestra versión MPI.

$$L_{ij} = \frac{a_{ij} - \sum_{k<j} L_{ik}L_{jk}}{L_{jj}}, \quad i > j$$

Right-looking: actualizar lo que falta

Al terminar una columna la difundimos. Con ella, cada proceso actualiza posiciones diferentes de la matriz restante. Esas actualizaciones de una misma etapa son independientes; la siguiente columna debe recibir antes todas las contribuciones que necesita.

$$S_{ik} \leftarrow S_{ik} - L_{ij}L_{kj}, \quad i \geq k > j$$

Por eso no decimos que las columnas completas sean independientes ni que left-looking no admita paralelismo. Right-looking nos ofrece una organización conveniente para repartir actualizaciones por propietario; su conveniencia práctica todavía debe medirse.

Cómo construimos entradas comparables

Generamos valores simétricos fuera de la diagonal y elegimos la diagonal mayor que la suma de sus valores absolutos. Para cualquier vector no nulo, al acotar cada producto cruzado con 2|x_i x_j| menor o igual que x_i²+x_j², resulta x transpuesta A x mayor o igual que la norma de x al cuadrado: A es definida positiva.

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

La semilla fija permite repetir la entrada. Trabajamos con esta familia de matrices diagonalmente dominantes; no afirmamos haber estudiado todas las condiciones numéricas posibles.

Código: src/cholesky.c, funciones generate_spd, cholesky_left y cholesky_right_buffered; src/cholesky_left_mpi.c y src/cholesky_mpi.c.

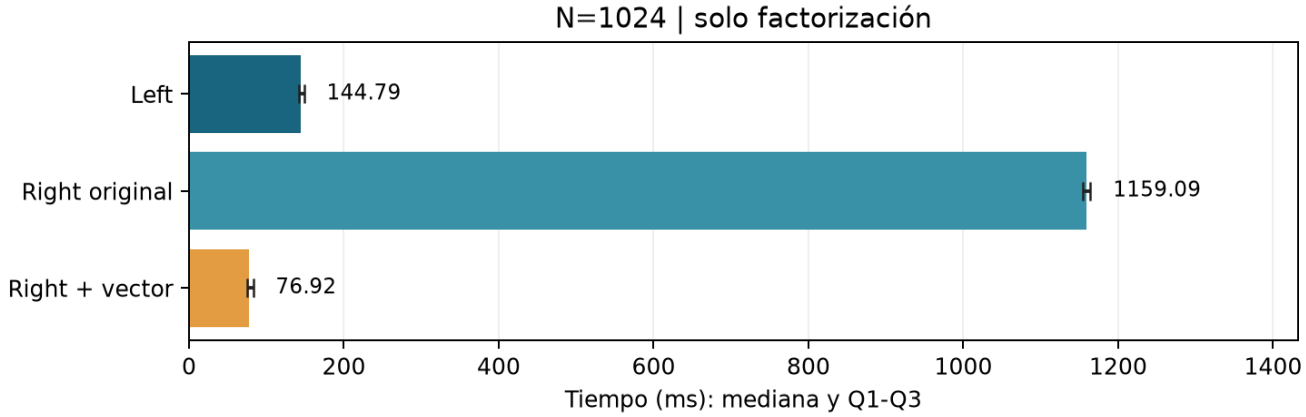
Etapa 1: la memoria cambia nuestra primera impresión

La pregunta inicial

Antes de repartir el trabajo, ¿qué implementación secuencial conviene usar como referencia? Probamos left-looking y right-looking sobre la misma entrada. Para N=1024, la primera campaña dio 137,69 ms para left y 1109,62 ms para right: el nombre del algoritmo no garantizaba un buen rendimiento.

La hipótesis y el cambio

En el almacenamiento por filas, recorrer una columna exige saltos. Copiamos la columna activa a un vector contiguo para reutilizarla en right-looking. Repetimos la comparación incluyendo las tres variantes en una nueva campaña; esa campaña es la que permite cuantificar el efecto del cambio.



memoria_20260921/resumen.csv: 12 repeticiones por variante; tiempo de copia incluido. Primera campaña: left_right_20260921, 7 repeticiones.

Qué decidimos y por qué

Conservamos right con vector: 76,92 ms frente a 1159,09 ms del right original en la misma campaña, unas 15,07 veces menos tiempo. También supera a left (144,79 ms). El cambio es compatible con mejor acceso y vectorización; no medimos fallos de caché para asignarles toda la causa.

La pregunta que abrió la siguiente etapa

Si el secuencial ya accede mejor a memoria, ¿cómo repartimos sus columnas entre procesos?

Archivos: src/compare_seq.c, src/compare_memory.c y src/cholesky.c. Elegimos una implementación concreta, no demostramos superioridad universal de right-looking.

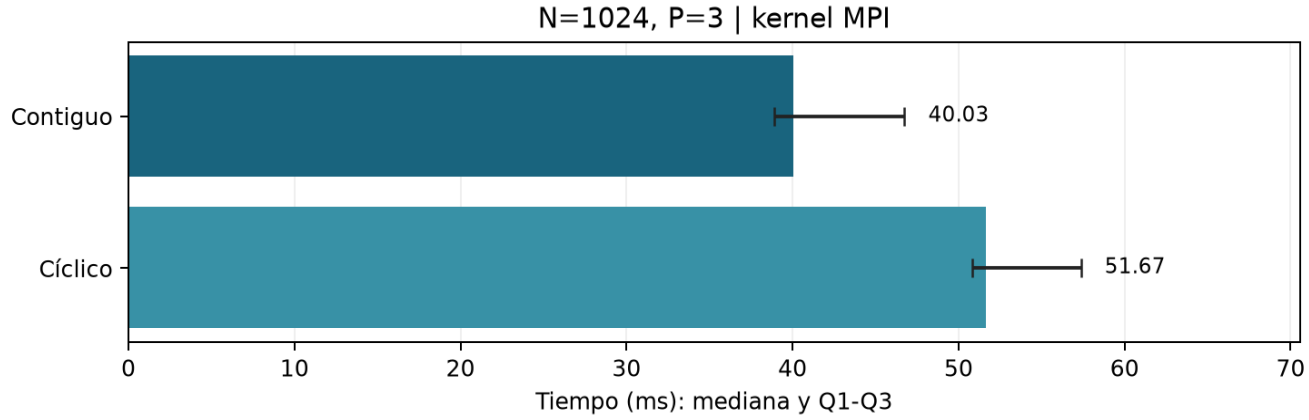
Etapa 2: equilibrar trabajo no alcanza

¿Columnas consecutivas o por turnos?

En el reparto contiguo cada proceso recibe un tramo de columnas. En el cíclico, las columnas se asignan por turnos: con P=3, el proceso 0 recibe 0,3,6…; el 1 recibe 1,4,7…; el 2 recibe 2,5,8… Las columnas de posiciones diferentes requieren distinta cantidad de actualizaciones, de modo que el cíclico promete distribuirlas mejor.

Qué probamos y qué observamos

Comparamos ambos repartos con 1,2,3 y 4 procesos. El primer código dio una sorpresa: en N=1024/P=3 el contiguo fue más rápido, aunque el reparto cíclico tenía un balance teórico más favorable.



mpi_20260921/resumen_mpi.csv: 8 repeticiones. Medición del kernel, sin generación ni distribución inicial.

Qué decidimos y por qué

El balance de actualizaciones no predice por sí solo el tiempo. Los índices y la forma de recorrer los datos también cambian entre repartos. Es una explicación que debemos investigar, no una medición aislada del costo de cada índice.

La pregunta que abrió la siguiente etapa

¿Podemos conservar el equilibrio del cíclico y simplificar a la vez el acceso a los datos que usa cada proceso?

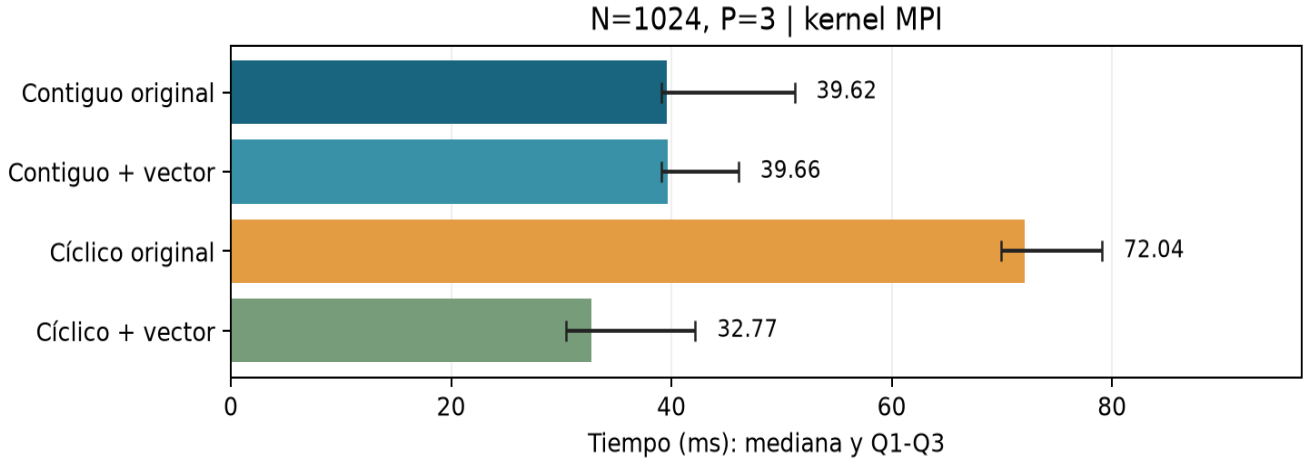
Archivos: src/cholesky_mpi.c y src/compare_mpi.c. Contiguo describe la propiedad de columnas; ambos repartos pueden almacenarse de manera contigua dentro de cada proceso.

Etapa 3: un vector adaptado a cada proceso

La idea que probamos

Después de difundir la columna, cada proceso extrae a un vector contiguo los valores que corresponden a sus propias columnas. El bucle de actualización consume ese vector local. Conservamos el algoritmo y el reparto; cambiamos la preparación y el acceso dentro del kernel.

Comparamos las cuatro combinaciones: reparto contiguo o cíclico, con el kernel original o con vector local. Eso permite comprobar si copiar ayuda a ambos repartos por igual.



mpi_local_20260921/resumen.csv: 8 repeticiones; copia local dentro del kernel. Los cuatro métodos pertenecen a la misma campaña.

Qué decidimos y por qué

El cíclico baja de 72,04 a 32,77 ms: una mejora de 2,20 veces entre variantes. Frente a la referencia secuencial contemporánea de 88,62 ms, el speedup útil es 2,70. Son dos cocientes distintos. Adoptamos cíclico con vector local como candidato, todavía sujeto a comparaciones posteriores.

La pregunta que abrió la siguiente etapa

¿A partir de qué tamaño el trabajo de cálculo alcanza para compensar la coordinación MPI?

Archivos: cholesky_mpi_local en src/cholesky_mpi.c y src/compare_mpi_local.c. La copia contigua apenas cambia su mediana; la mejora no debe generalizarse a todo acceso ni a toda arquitectura.

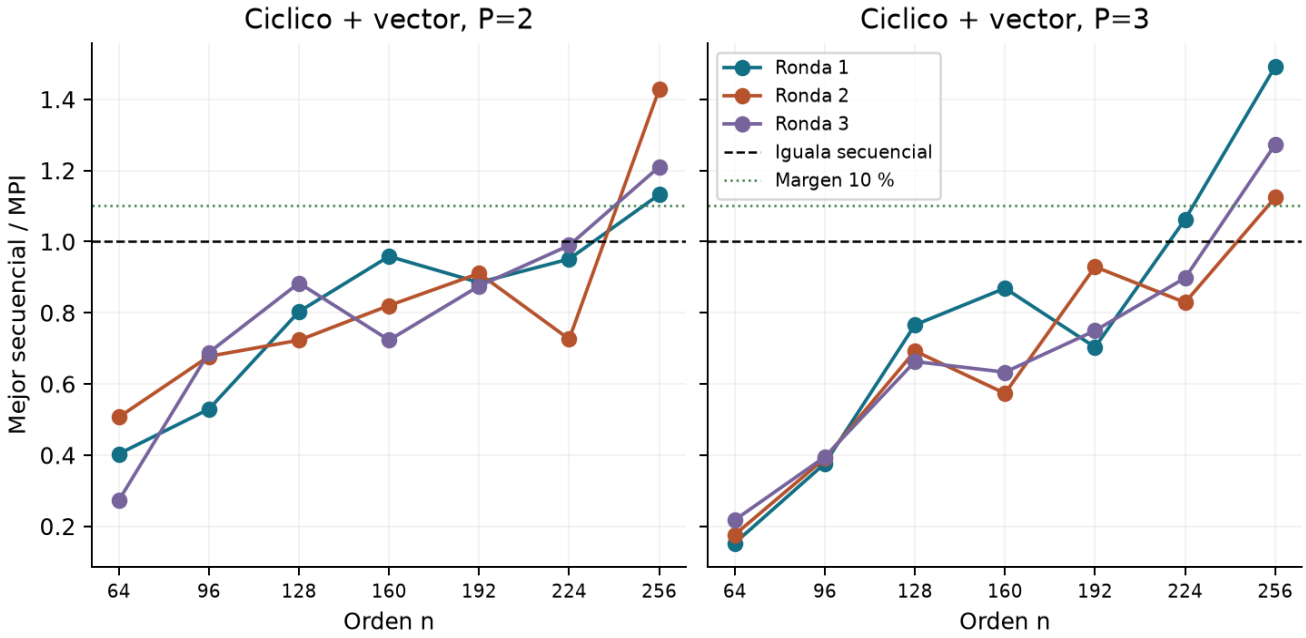
Etapa 4: buscar el cruce, no inventar un umbral universal

Qué queríamos saber

Usar más procesos agrega coordinación y comunicación. Para matrices pequeñas puede costar más que resolver el problema secuencialmente. Probamos N=64,96,128,160,192,224,256 con P=2 y P=3, en tres rondas.

El criterio fijado antes de concluir

Exigimos speedup superior a 1,10 y Q3 del tiempo MPI menor que Q1 del secuencial en las tres rondas. Es un criterio descriptivo conservador, no una prueba de significación estadística.



umbral_20260922: tres rondas; 2016 tiempos MPI y 756 secuenciales. El reloj mide factorización, sin generar, distribuir ni reunir.

Qué decidimos y por qué

N=256/P=3 fue el primer caso ensayado que cumplió el criterio en las tres rondas. P=2 no lo cumplió de forma consistente. Es un cruce observado para este kernel, esta máquina y este reloj; no una regla N mayor o igual que 256 para cualquier MPI.

La pregunta que abrió la siguiente etapa

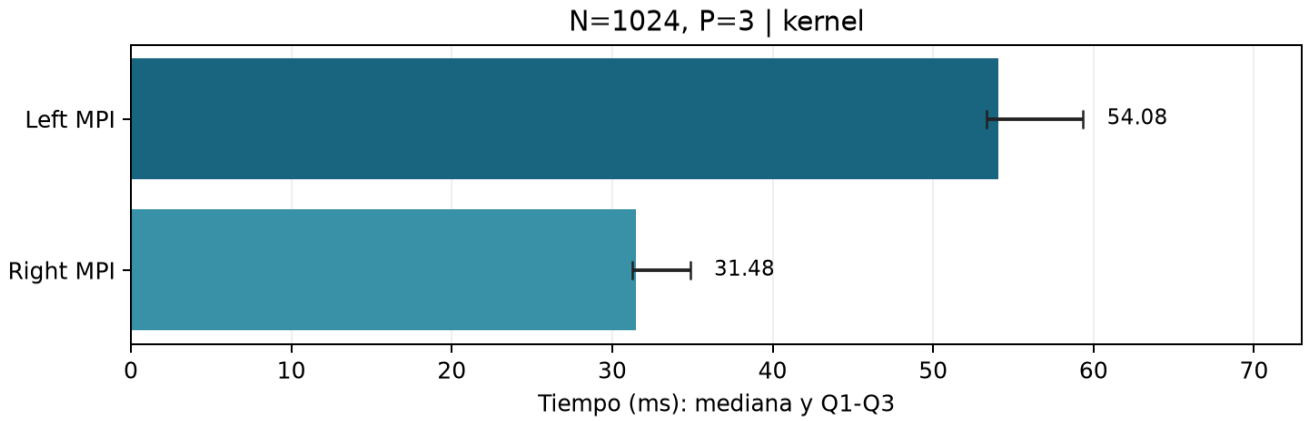
Antes de ampliar el reloj, ¿la ventaja de right sigue en pie si también implementamos left con MPI?

Archivos: scripts/medir_umbral.sh, scripts/analizar_umbral.py; evidencia en umbral.csv, rondas.csv y conclusiones.json. No hubo comunicación entre nodos: no atribuimos el cruce a latencia de red del clúster.

Etapa 5: comparar left y right también en MPI

La duda que faltaba resolver

La elección de right no podía apoyarse en que left fuera imposible de paralelizar. Implementamos left repartiendo entre procesos las contribuciones de las columnas anteriores y combinándolas mediante reducciones. Usamos las mismas entradas y el mismo reparto cíclico.



left_right_mpi_20260922/resumen.csv: 8 repeticiones. Right obtuvo menor mediana en las 20 configuraciones comparadas.

Qué coordinación realiza nuestro código

Left ejecuta N reducciones de diagonal, N-1 reducciones vectoriales y N difusiones del pivote: 3N-1 colectivas en total, de las que 2N-1 son Reduce. Right difunde la columna una vez por etapa: N Bcast. Ese conteo es del código implementado; no es una obligación de cualquier Cholesky left-looking.

Qué decidimos y por qué

Right tarda 31,48 ms frente a 54,08 ms de left en este caso, una relación de 1,72. Su organización de comunicación y trabajo local respalda conservarlo. No demostramos saturación de red ni que toda variante left sea inferior.

La pregunta que abrió la siguiente etapa

Hasta ahora priorizamos el kernel. ¿Se mantiene la ventaja al repartir la matriz de entrada y reunir el resultado?

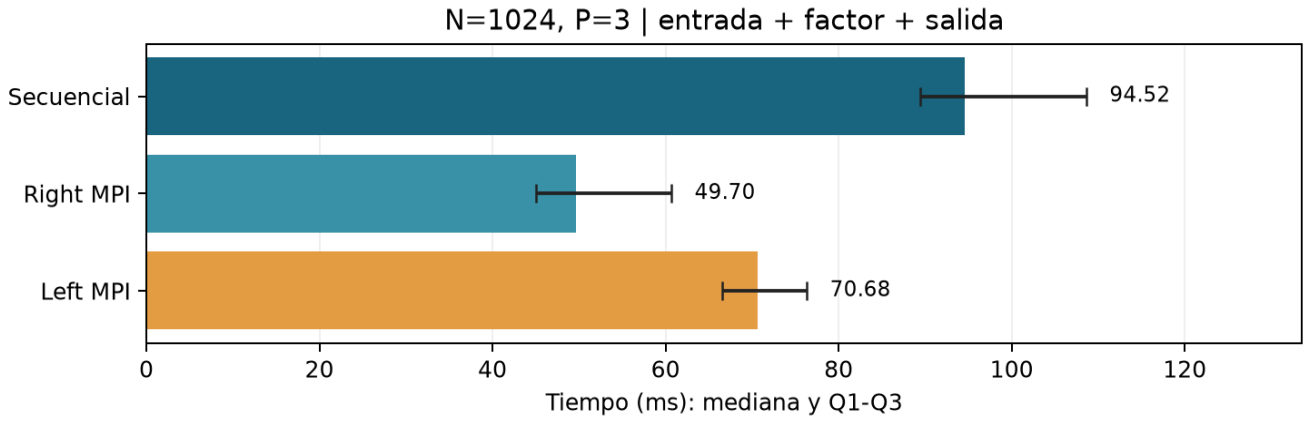
Archivos: src/cholesky_left_mpi.c, src/cholesky_mpi.c y src/compare_mpi_algorithms.c. La comparación evalúa estas implementaciones completas, no solo el número de colectivas.

Etapa 6: incluir entrada y salida cambia la respuesta

La pregunta se acerca al uso del programa

Además de factorizar, ahora contamos el empaquetado y distribución inicial, y la reunión del factor. La matriz ya existe antes del reloj: en esta etapa todavía no se incluye su generación ni el arranque de MPI.

$$T = T_{entrada} + T_{factor} + T_{salida}$$



tiempo_total_20260922/resumen.csv: 12 repeticiones. Totales medidos directamente; no son sumas de medianas.

Right necesita 49,70 ms frente a 94,52 ms secuenciales: speedup 1,90. El cociente limitado a la factorización de esta misma campaña era 2,41. Incluir fases adicionales reduce la ventaja percibida.

Qué decidimos y por qué

N=512 fue el primer tamaño ensayado con una mejora total clara en esa campaña. La pregunta cambió y el cruce observado también. Mantenemos right, pero no llamamos a este intervalo el tiempo externo completo del programa.

La pregunta que abrió la siguiente etapa

Si también debemos generar los datos, ¿conviene que nazcan completos en el proceso 0 o que cada proceso genere su parte?

Archivos: src/compare_mpi_total.c y scripts/comparar_tiempo_total.sh. Seguimos en una sola máquina; Scatterv y Gatherv no implican que ya se haya medido una red entre nodos.

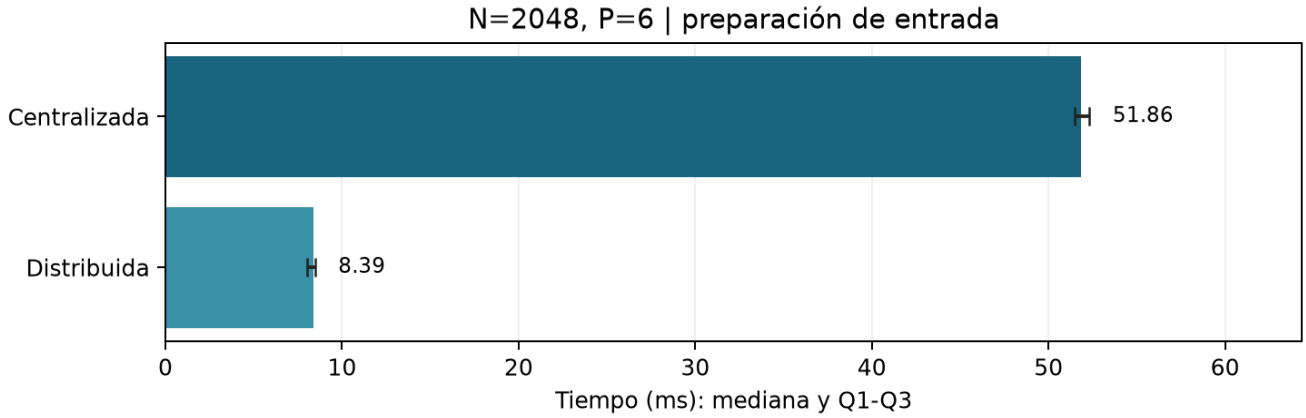
Etapa 7: generar donde vamos a calcular

Qué cambió en el planteo

La consulta docente reforzó la necesidad de considerar todo el proceso, incluida la generación. Por eso investigamos dos escenarios válidos: generar en el proceso 0 y distribuir, o generar directamente las columnas locales. No suponemos que todas las aplicaciones reales deban iniciar con datos distribuidos.

Cómo mantuvimos exactamente la misma entrada

Usamos los índices de cada par simétrico para recuperar el mismo valor de la secuencia pseudoaleatoria sin recorrer sus valores anteriores. Cada proceso puede reconstruir sus columnas sin intercambiar los elementos generados. La diagonal conserva la suma de valores absolutos y el mismo orden de acumulación.



generacion_local_20260922/resumen.csv: 12 repeticiones por método. Preparación solamente: no es tiempo de toda la aplicación.

Qué decidimos y por qué

La preparación baja de 51,86 a 8,39 ms: una relación de 6,18 entre métodos con el mismo P. La generación distribuida pasa a ser candidata para este escenario. No es un speedup por pasar de uno a seis procesos.

La pregunta que abrió la siguiente etapa

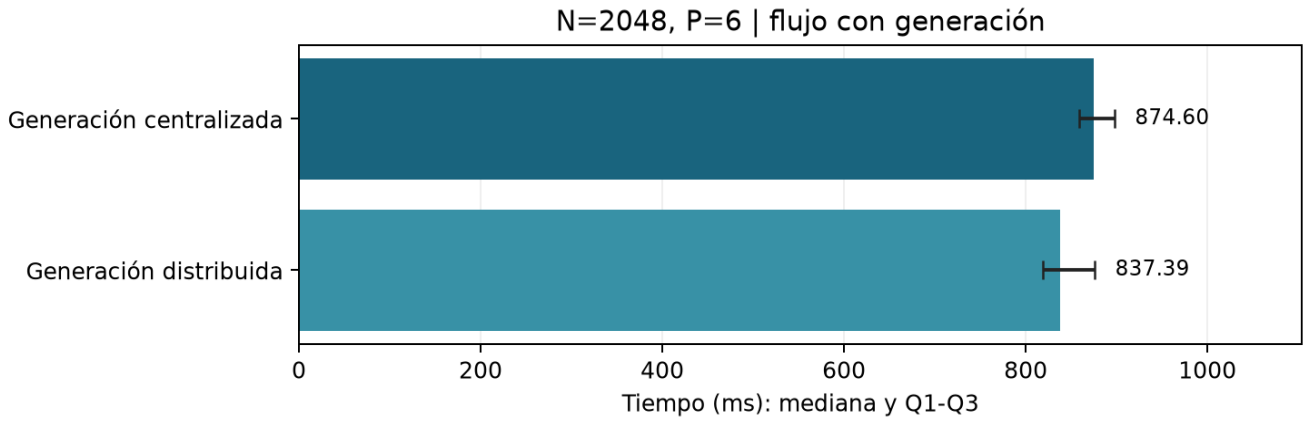
¿Reducir mucho una fase corta también reduce mucho el tiempo de toda la aplicación?

Archivos: indexed_value y generate_spd_columns en src/cholesky.c; src/compare_generation.c. Se comprobó coincidencia exacta de las matrices generadas en ambos caminos.

Etapa 8: una mejora grande puede ahorrar poco en total

La prueba que faltaba

Integramos la generación al flujo completo: reserva, generación, distribución cuando corresponde, factorización y recolección. Comparamos ambas preparaciones obteniendo el mismo resultado completo en rango 0. Validación y arranque siguen fuera del reloj de aplicación.



aplicacion_local_20260922/resumen.csv: 6 repeticiones por método. Las barras describen variabilidad; no prueban significación estadística.

El total pasa de 874,60 a 837,39 ms: una reducción de mediana de 4,25%. El cálculo ocupa alrededor del 91,1% del tiempo en el flujo distribuido de ese caso. Por eso el ahorro del generador tiene un efecto limitado sobre el total.

Qué decidimos y por qué

Conservamos la generación distribuida para el escenario del candidato. El siguiente esfuerzo de optimización debería mirar la fase dominante, sin olvidar que el objetivo es reducir el total medido. Una mejora local de una fase no garantiza una mejora proporcional de la aplicación.

La pregunta que abrió la siguiente etapa

¿Cambiar el tamaño de los grupos de columnas asignados a cada proceso mejora el flujo completo?

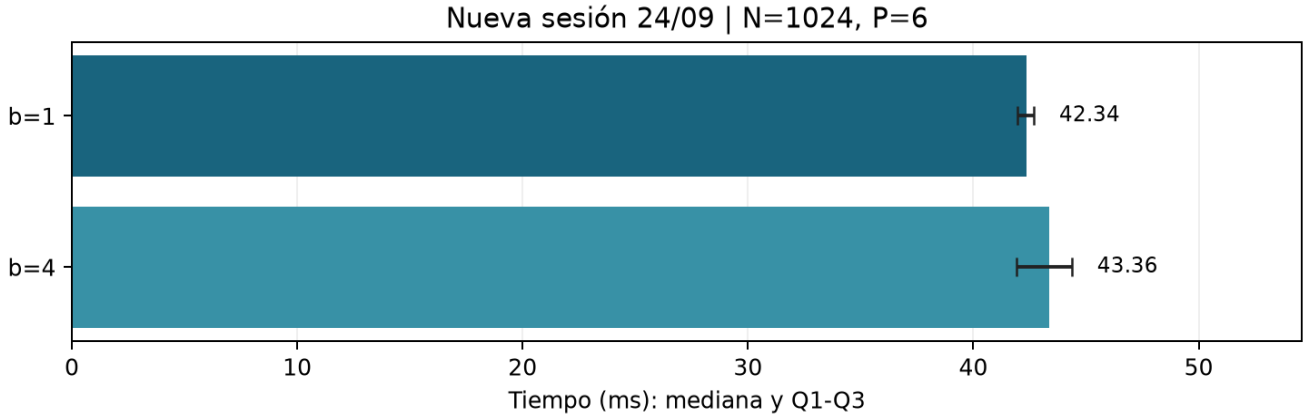
Archivos: src/compare_application.c y scripts/analizar_aplicacion.py. El porcentaje de factorización proviene de proporciones por ejecución resumidas; no se obtiene sumando medianas de fases.

Etapa 9: agrupar columnas y volver a comprobar

La alternativa

Probamos grupos b=1,2,4,8,16,32,64 columnas. Seguimos en distribución 1D: cambia quién posee las columnas, no introducimos Cholesky por bloques ni operaciones matriz-matriz. b=1 reproduce el reparto cíclico una a una.

b=4 pareció favorable en N=1024/P=6 y una repetición inicial mostró alrededor de 10% de ventaja. Antes de adoptarlo, lo contrastamos en otra sesión. Allí esa ventaja desapareció.



precluster_20260924/resumen.csv: 8 repeticiones por reparto; flujo con generación. Campaña exploratoria anterior: bloques_local_20260922.

En N=2048 también se observan medianas de 767,99 ms para b=1 y 798,87 ms para b=4. El piloto con N=3072 solo tuvo dos repeticiones: demuestra factibilidad, no estabilidad estadística.

Qué decidimos y por qué

Mantenemos b=1 como referencia simple. La ventaja de b=4 no fue consistente entre sesiones; no afirmamos equivalencia estadística ni que b=1 sea siempre óptimo.

La pregunta que abrió la siguiente etapa

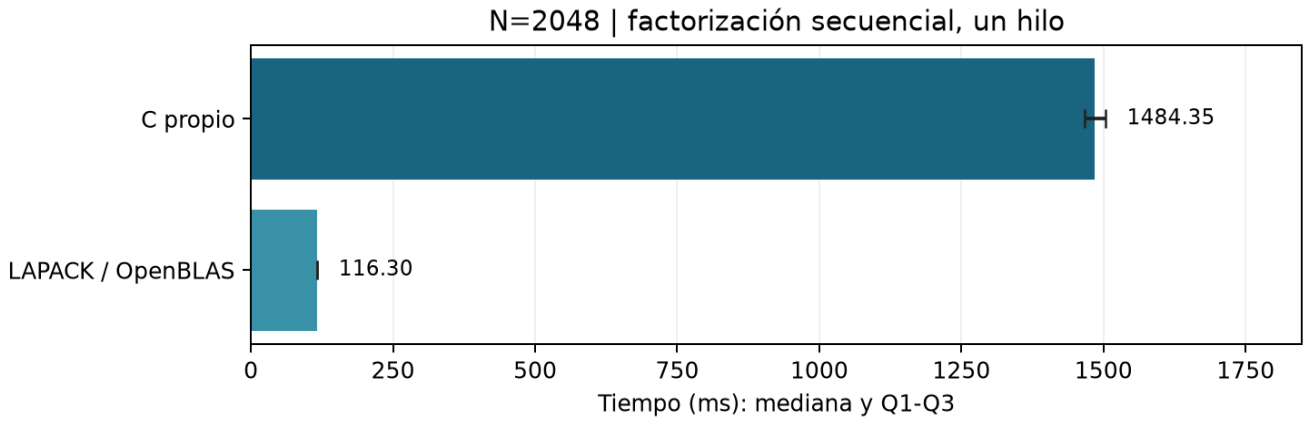
¿Cuánto margen de rendimiento queda frente a una biblioteca numérica optimizada?

Archivo: src/compare_blocks.c. La asignación dinámica queda como discusión: agregaría coordinación para dependencias conocidas. No fue implementada ni medida, por lo que no se descarta universalmente.

Etapa 10: LAPACK como referencia, no como reemplazo

Qué quisimos dimensionar

Comparamos nuestro right-looking secuencial con LAPACK mediante LAPACKE y OpenBLAS de un hilo. Ambos reciben la misma entrada, en un ejecutable sin MPI y con afinidad a CPU 0. Esta comparación no mide aceleración por aumentar procesos.



precluster_20260924/resumen.csv: 6 repeticiones. La configuración de un hilo quedó comprobada por el banco de prueba.

El factor pasa de 1484,35 ms a 116,30 ms, una relación de 12,76. Al incluir reserva y generación, los totales son 1527,24 y 156,85 ms: relación 9,74. Es evidencia de margen de mejora, no una medición del aporte aislado de SIMD, caché o bloques.

Qué decidimos y por qué

Conservamos el MPI propio para investigar la distribución del algoritmo y usamos la biblioteca como referencia externa. Implementar un Cholesky propio por bloques abriría otra etapa; no es necesario atribuirle ya los resultados de LAPACK.

La pregunta que abrió la siguiente etapa

¿Cómo reunimos las decisiones adoptadas en un programa candidato claro, separado de los bancos experimentales?

Archivo: src/compare_lapack.c; Makefile, target lapack. Se llama LAPACKE_dpotrf_work. N=3072 tiene solo dos observaciones por variante y se trata como exploratorio.

Etapa 11: construir un candidato que podamos explicar

Qué consolidamos

La investigación produjo bancos de comparación, no diez programas finales. Reunimos la elección vigente en src/main_candidate.c. Con USE_MPI compila el candidato paralelo; sin esa definición compila una referencia secuencial autónoma. No cambiamos el núcleo MPI que habíamos seleccionado.

Parte	Decisión incorporada
Cálculo	Right-looking escalar con vector local.
Propiedad	Columnas 1D cíclicas, b=1.
Entrada MPI	Cada proceso genera sus columnas con semilla 2026.
Salida	Factor completo en memoria del rango 0; CSV de métricas. No se escribe L a disco.
Control	Residuo proyectado, valores finitos y diagonal positiva; residuo completo para N menor o igual a 128.

Qué comprobamos

Probamos N=1,2,7,31,128 en secuencial y con P=1,2,3,4,6: 30 configuraciones, incluidas algunas con más procesos que columnas. Todas pasaron. Las pruebas previas de factores conocidos y errores también pasaron. Rechazamos parámetros inválidos con código 2; la validación fallida devuelve 1; un fallo de reserva aborta con 3. No garantizamos un único código para cualquier fallo del runtime MPI.

Las proyecciones de costo cuadrático permiten comprobar casos grandes sin reconstruir LL transpuesta completa. No son un certificado de corrección para toda matriz. Los casos pequeños mantienen una comprobación completa y referencias conocidas.

Qué decidimos y por qué

Disponemos de un ejecutable reproducible para el escenario elegido. Conservamos los bancos anteriores como evidencia; el candidato no incorpora todas las variantes.

La pregunta que abrió la siguiente etapa

¿Cuánto gana este programa frente a un secuencial autónomo cuando medimos la aplicación de principio a resultado?

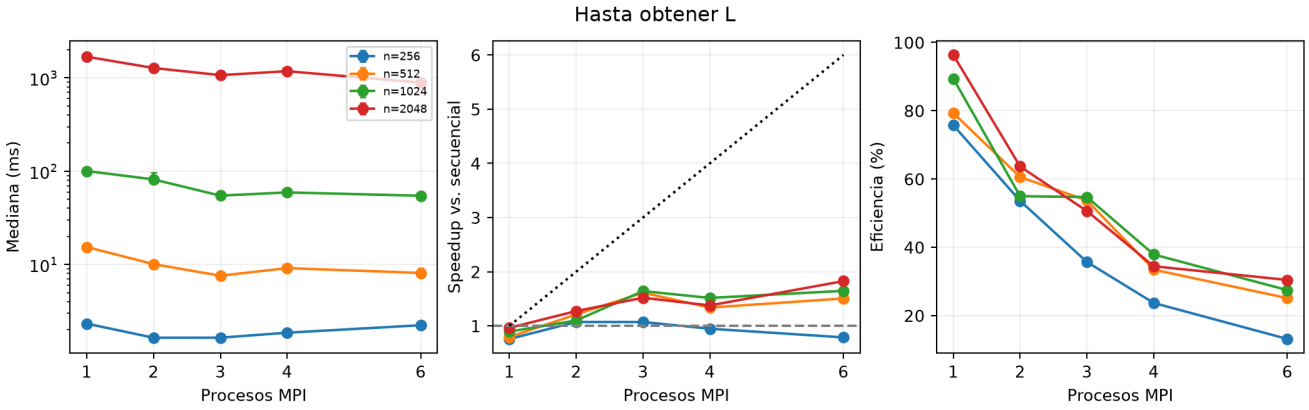
Archivos: src/main_candidate.c, src/cholesky.c, src/cholesky_mpi.c; scripts/verificar_interfaz_candidato.sh. Evidencia: cierre_local_20260924/tests y build_tests.log. La salida contiene cabecera CSV y una fila de resultados.

Etapa 12: medir aceleración y aprovechamiento

Cómo construimos una comparación principal

Usamos el secuencial autónomo y MPI con P=1,2,3,4,6 para N=256,512,1024,2048. Cada observación es un lanzamiento nuevo; descartamos la primera ronda y medimos siete más. Son 168 observaciones útiles y 24 de preparación. El orden de métodos rota entre rondas.

$$S(P) = \frac{\text{mediana}(T_{seq})}{\text{mediana}(T_{MPI,P})}, \qquad E(P) = S(P)/P$$



cierre_local_20260924/resumen.csv: 7 repeticiones. Reloj de aplicación: reserva, generación, factorización y reunión; excluye validación y arranque MPI.

En N=2048/P=6: 1624,85 ms secuenciales frente a 890,19 ms MPI. El speedup es 1,83 y la eficiencia 30,4%. La ganancia existe, pero seis procesos no producen seis veces la velocidad.

Qué decidimos y por qué

Informamos tiempo, speedup y eficiencia juntos. La escalabilidad fuerte se estudia fijando N y aumentando P; aumentar N con P fijo estudia otro aspecto, la amortización del costo paralelo. No usamos esa segunda observación como prueba de escalabilidad fuerte.

La pregunta que abrió la siguiente etapa

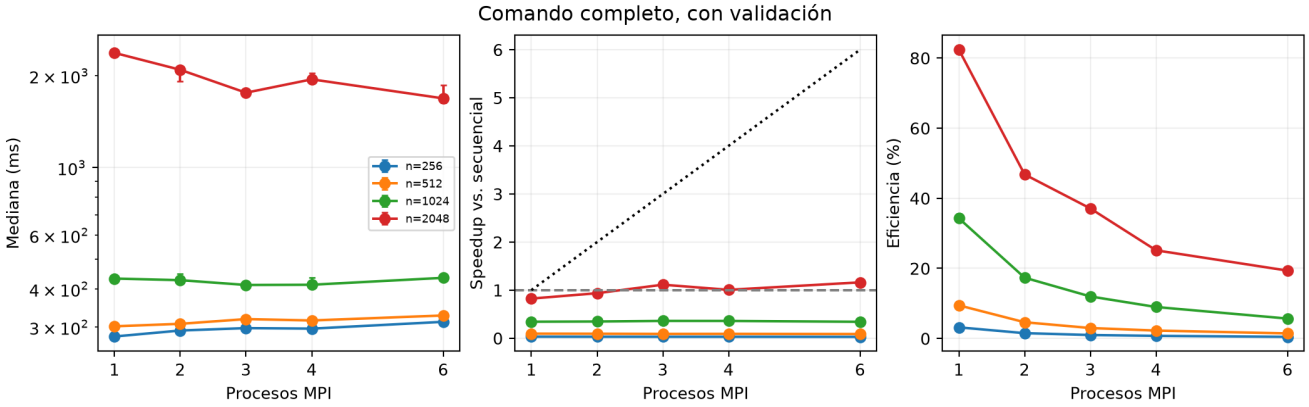
¿La ventaja hasta obtener L se mantiene al medir también lanzamiento, validación y cierre?

Archivos: scripts/cierre_local.sh y scripts/analizar_cierre.py. WSL: seis núcleos físicos, MPI ligado a núcleos; secuencial en CPU 0. Los máximos locales se guardan aparte del reloj de rango 0.

Etapa 13: el reloj externo cambia el veredicto

Un segundo reloj para otra pregunta

measure_command.c usa un reloj monotónico desde antes de crear el proceso hijo hasta que termina el comando. Para MPI mide mpirun; para el secuencial, taskset. Incluye arranque, aplicación, regeneración para validar, residuos, salida CSV, liberación y cierre. No incluye iniciar WSL ni una cola del clúster.



Mismas 168 observaciones de la etapa 12; cambia la frontera del reloj, no el conjunto de ejecuciones. Cuartiles en la gráfica de tiempos.

N=2048, P=6	Secuencial	MPI	Speedup
Hasta L	1624,85 ms	890,19 ms	1,83
Comando completo	1957,48 ms	1685,41 ms	1,16

Con N=1024/P=6, la aplicación mejora pero el comando completo tarda 434,99 ms frente a 148,30 ms secuenciales: S externo=0,34. Las 192 ejecuciones de campaña fueron válidas; el residuo proyectado máximo fue $1{,}61 \times 10^{-15}$.

Qué decidimos y por qué

La ganancia depende de qué trabajo medimos. La validación crece con N y no es un costo fijo; el tiempo externo adicional tampoco mide solo arranque o comunicación. No deducimos un umbral universal N mayor o igual que 2048 para el clúster.

La pregunta que abrió la siguiente etapa

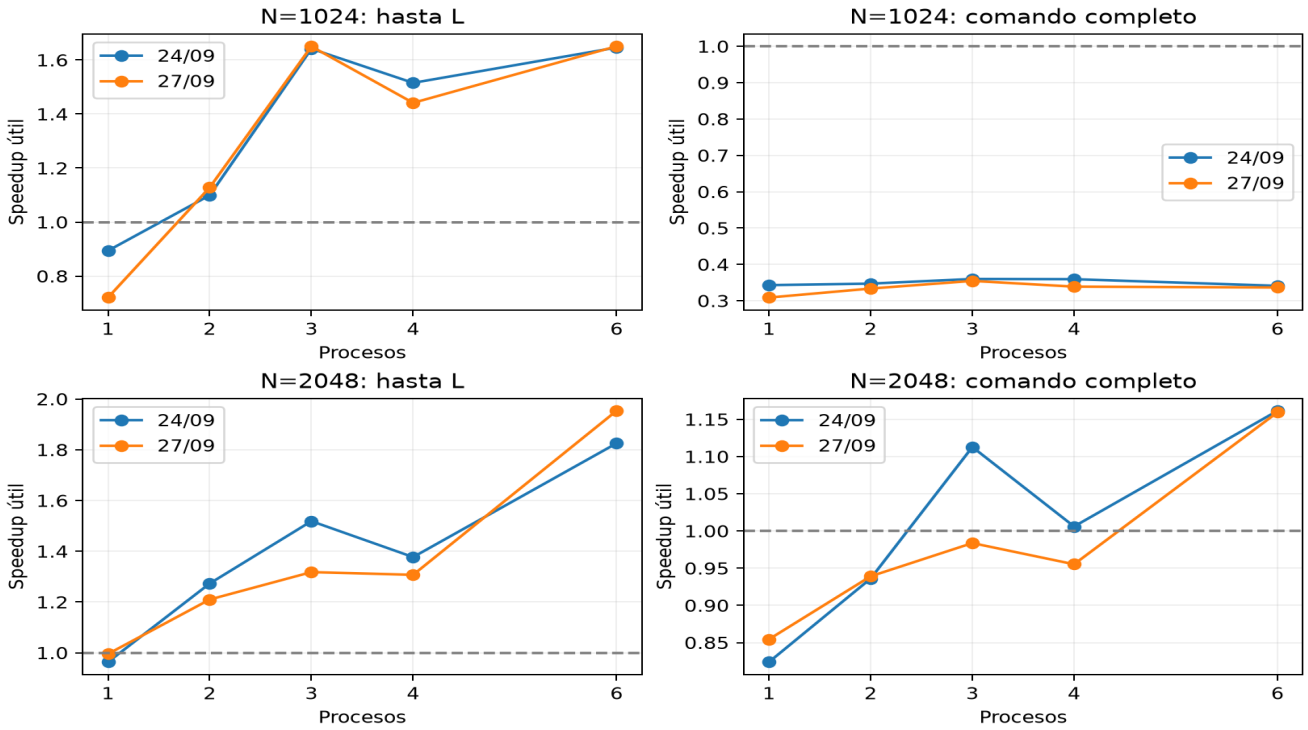
La ventaja externa de 1,16 es modesta. ¿Se repite en otra sesión con exactamente el mismo candidato?

Archivos: src/measure_command.c y src/main_candidate.c. Los totales se miden directamente; el resto externo mezcla varios costos y no permite atribuir causalidad a uno de ellos.

Etapa 14: repetir antes de sostener una conclusión

El control de otra sesión

El 27/09 repetimos el protocolo sin cambiar código ni script: las ocho fuentes congeladas coinciden byte por byte. Nuevamente, 192 lanzamientos válidos y 168 tiempos medidos. Calculamos cada speedup con su referencia secuencial de esa sesión; no mezclamos observaciones entre días.



Comparación de cocientes de medianas entre 24/09 y 27/09. Sin intervalos de confianza; las muestras y cuartiles se conservan por separado.

En N=2048/P=6, el speedup de aplicación pasa de 1,83 a 1,95; el externo queda en 1,16 redondeado en ambas sesiones. En cambio, con P=3 el externo pasa de 1,11 a 0,98: esa pequeña ventaja no se repite.

Qué decidimos y por qué

Conservamos el candidato como base local identificable. Hay evidencia repetida a favor de seis procesos en ese caso, pero no de una aceleración fija ni de superioridad para todo tamaño. No aislamos temperatura, frecuencia o carga para explicar las variaciones.

La pregunta que abrió la siguiente etapa

¿Qué cambia cuando aumentamos las matrices y pasamos de procesos de una máquina a procesos en distintos nodos?

Evidencia: cierre_local_20260927/comparacion_sesiones.csv y comparacion_fuentes.json. Esta es la etapa 14 efectivamente respaldada por los archivos disponibles.

Qué podemos defender al cerrar la etapa local

La decisión que tiene respaldo

El candidato es Cholesky right-looking en C con MPI, reparto 1D cíclico b=1, vector local, generación distribuida y factor reunido en rango 0. Lo elegimos por el comportamiento de nuestras implementaciones medidas, no porque las demás alternativas sean incapaces de paralelizarse.

Lo aprendido	Lo que no demostramos
El acceso a memoria cambió fuertemente los tiempos.	No aislamos porcentajes debidos a caché, SIMD o indexación.
La comunicación y la preparación cambian la ganancia útil.	No fijamos un umbral universal de N o una cantidad óptima de procesos.
La repetición confirmó unas ventajas y debilitó otras.	Dos sesiones no prueban estabilidad en cualquier máquina.
LAPACK muestra margen importante de mejora.	No medimos por separado cada optimización interna de esa biblioteca.
El candidato pasa los casos numéricos ejecutados.	No cubrimos todas las matrices SPD ni todas sus condiciones numéricas.

Qué significa cerrar esta parte del proyecto

Cerramos un alcance concreto y defendible: construir el algoritmo, comparar decisiones, integrar el programa y medirlo con dos relojes. Versionar el candidato permite saber qué código produjo cada dato. No impide corregir errores; una corrección posterior se valida y se registra como otra versión.

La siguiente investigación

En el clúster habrá que compilar y validar, elegir tamaños mediante pilotos, medir el secuencial en ese mismo entorno y comparar MPI dentro de un nodo y entre nodos. Se conservarán tiempos absolutos, speedup, eficiencia, dispersión y condiciones de ejecución. Los tiempos de la PC no serán denominadores de speedup del clúster.

Quedan como extensiones posibles un Cholesky propio por bloques, otras familias SPD, OpenMP, un reparto 2D y entrada/salida real de matrices. No forman parte de lo demostrado aquí.

Reproducción del documento: docs/generar_relato_investigacion.py. Las figuras nuevas leen los resúmenes de las campañas; las figuras reutilizadas mantienen su protocolo original. Manifiesto: docs/relato_investigacion_fuentes.json. Los informes anteriores permanecen intactos.